

Week 9 — Day 3

Guided Lab: Model an Online Store

Lab explanation + complete solution + Exit Ticket

Lab instruction: Define models only. Do not run migrations yet.

Inside this guide

Requirements 1-8 • Step-by-step solution • related_name • on_delete justifications • Complete models.py • Exit Ticket • Final checklist • Quick Review

Lab status: Complete according to the requirements shown in the lab. No extra features or migrations are included.

Python Bootcamp • Study Notes

1. Lab Requirements

#	Requirement
1	Category has an optional parent Category.
2	Product belongs to one Category.
3	User has one CustomerProfile.
4	Order belongs to one User.
5	Order connects to many Products through OrderItem.
6	OrderItem stores quantity and unit_price.
7	Every relationship has a clear related_name.
8	Justify each on_delete choice in one sentence.

2. Step 1 — Category

Category can optionally point to another Category, so we use a self-referencing ForeignKey.

```
parent = models.ForeignKey(
    "self", on_delete=models.SET_NULL,
    null=True, blank=True, related_name="children"
)
```

Why: SET_NULL keeps the child category and clears its parent link. null=True allows database NULL; blank=True makes it optional.

3. Step 2 — Product

Product belongs to one Category; the ForeignKey is therefore on Product.

```
category = models.ForeignKey(
    Category, on_delete=models.PROTECT,
    related_name="products"
)
```

Why: PROTECT prevents deleting a category while products still depend on it.

4. Step 3 — CustomerProfile

Each User has one CustomerProfile, so this is One-to-One.

```
user = models.OneToOneField(
    User, on_delete=models.CASCADE,
    related_name="customer_profile"
)
```

Why: CASCADE removes the profile when its user is deleted.

5. Step 4 — Order belongs to User

One User can have many Orders, so Order stores the ForeignKey.

```
user = models.ForeignKey(
    User, on_delete=models.CASCADE,
    related_name="orders"
)
```

Why: CASCADE removes the user's orders when that user is deleted.

6. Step 5 — Order ↔ Product

An Order can contain many Products and a Product can appear in many Orders. The relationship needs extra data, so the lab uses OrderItem as the through model.

```
products = models.ManyToManyField(
    Product, through="OrderItem",
    related_name="product_orders"
)
```

Why through? quantity and unit_price belong to a specific Product inside a specific Order.

7. Step 6 — OrderItem

OrderItem connects one Order to one Product and stores the line-item details.

```
order = models.ForeignKey(
    Order, on_delete=models.CASCADE,
    related_name="items"
)

product = models.ForeignKey(
    Product, on_delete=models.PROTECT,
    related_name="order_items"
)

quantity = models.PositiveIntegerField()
unit_price = models.DecimalField(
    max_digits=10, decimal_places=2
)
```

Why: CASCADE for Order → OrderItem because items belong to the order. PROTECT for Product → OrderItem to preserve referenced order history.

8. Step 7 — related_name

The final reverse-access names are:

Relationship	related_name
Category → parent	children
Product → Category	products
CustomerProfile → User	customer_profile
Order → User	orders
Order → Product	product_orders
OrderItem → Order	items
OrderItem → Product	order_items

Important: product_orders avoids a reverse-name collision with the User → Order name, orders.

9. Step 8 — on_delete Justifications

Category → Category — SET_NULL

If a parent category is deleted, the child category should remain but its parent relationship should be cleared.

Product → Category — PROTECT

A category should not be deleted while products still depend on it.

CustomerProfile → User — CASCADE

If a user is deleted, their customer profile should also be deleted.

Order → User — CASCADE

If a user is deleted, their orders should also be deleted.

OrderItem → Order — CASCADE

If an order is deleted, its order items should also be deleted because they belong to that order.

OrderItem → Product — PROTECT

A product should not be deleted while existing order items still reference it.

10. Complete models.py

The complete lab solution:

```
from django.db import models
from django.contrib.auth.models import User

class Category(models.Model):
    name = models.CharField(max_length=100)
    parent = models.ForeignKey(
        "self", on_delete=models.SET_NULL,
        null=True, blank=True, related_name="children"
    )

class Product(models.Model):
    name = models.CharField(max_length=150)
    category = models.ForeignKey(
        Category, on_delete=models.PROTECT,
        related_name="products"
    )

class CustomerProfile(models.Model):
    user = models.OneToOneField(
        User, on_delete=models.CASCADE,
        related_name="customer_profile"
    )
```

```
class Order(models.Model):
    user = models.ForeignKey(
        User, on_delete=models.CASCADE,
        related_name="orders"
    )
    products = models.ManyToManyField(
        Product, through="OrderItem",
        related_name="product_orders"
    )

class OrderItem(models.Model):
    order = models.ForeignKey(
        Order, on_delete=models.CASCADE,
        related_name="items"
    )
    product = models.ForeignKey(
        Product, on_delete=models.PROTECT,
        related_name="order_items"
    )
    quantity = models.PositiveIntegerField()
    unit_price = models.DecimalField(
        max_digits=10, decimal_places=2
    )
```

Migration status: Do not run makemigrations or migrate yet. The lab explicitly says to define models only.

11. Exit Ticket

Question: Why does quantity belong in OrderItem rather than Product or Order?

Answer: quantity belongs in OrderItem because it describes how many units of a specific product were included in a specific order. It is not a permanent property of the Product and it does not describe the entire Order.

Don't Confuse

Product = the product itself.

Order = the overall purchase.

OrderItem = one product line inside that purchase, including quantity and unit_price.

12. Final Lab Checklist

- ✓ Requirements 1-8 completed.
- ✓ Category has an optional parent.
- ✓ Product belongs to one Category.
- ✓ CustomerProfile is One-to-One with User.
- ✓ Order belongs to one User.
- ✓ Order ↔ Product uses Many-to-Many through OrderItem.
- ✓ OrderItem stores quantity and unit_price.
- ✓ Every relationship has a clear related_name.
- ✓ Every on_delete choice has a one-sentence justification.
- ✓ Exit Ticket answered.
- ✓ Migrations were NOT run.

LAB COMPLETE. The solution follows the requirements shown in the lab. No extra work is required at this stage.

Quick Review

1. ForeignKey in one-to-many? — The many side.
2. Why OrderItem as through model? — The relationship needs extra data.
3. null=True vs blank=True? — Database NULL vs validation/forms.
4. Why SET_NULL for Category.parent? — Keep the child and clear the parent link.
5. Why PROTECT for Product.category? — Prevent deleting a category that existing products depend on.